BACKEND AUDIT & REMEDIATION

Home Insurance Management System

Nine blockers found in the Spring Boot microservices backend, what was wrong with each, how it was fixed, and the live evidence that it is now closed.

DATE
9 September 2026

SCOPE
7 modules · 33 files changed · 20 added

STACK
Spring Boot 4.0.7 · Cloud 2025.1.2 · Java 17/24

VERIFICATION
104 live API checks

All 9 blockers cleared

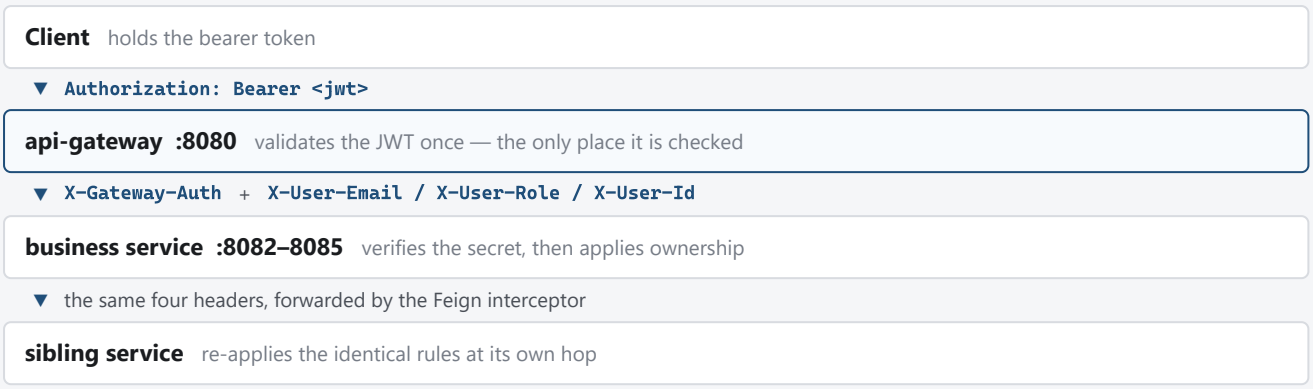
104 checks passed, 0 failed, against the full stack running on Eureka + gateway + 5 services with MySQL.

§1 Summary

#	BLOCKER	SEVERITY	EFFECT BEFORE THE FIX	STATUS
1	No role check on claim decisions	Critical	A customer could approve their own claim	Cleared
2	No ownership checks	Critical	Any user could read and overwrite any other user's records	Cleared
3	Services open on their own ports	Critical	Ports 8082–8085 served all data with no token at all	Cleared
4	Auth errors all returned 401	High	"Email already taken" reported as "Unauthorized"	Cleared
5	Duplicate customer profile	High	Broke `/customers/user/{id}` permanently with a 500	Cleared
6	Bad enum/type values → 500	Medium	Client input errors surfaced as server faults	Cleared
7	No business rules on claims	High	Future dates, pre-policy dates, loss above cover, unlimited duplicates	Cleared
8	No business rules on payments	High	₹1 settled a ₹35,000 premium; unlimited overpayment	Cleared
9	Colliding policy/claim numbers	Low	Random 6-digit against a unique column → occasional 500	Cleared

§2 The trust layer — blockers 1, 2 and 3 together

These three could not be fixed independently. Blockers 1 and 2 are authorization rules that rely on the `X-User-*` headers the gateway sets. But while blocker 3 was open, anyone who could reach port 8084 directly could simply send `X-User-Role: ADMIN` and walk straight past those rules. So all three share one mechanism.



Three parts. (1) auth-service now puts `userId` in the JWT, and the gateway forwards it as `X-User-Id` beside email and role — set, not appended, so anything the client sent under those names is discarded. (2) The gateway stamps `X-Gateway-Auth` from a new shared secret; every business service has a `GatewayAuthFilter` that rejects requests without it, which is what makes the identity headers trustworthy. (3) Feign calls forward both the secret and the caller's identity, so each hop in `claim → policy → customer` re-applies the same ownership rules.

Ownership chain. Nothing stores an owner column; ownership is derived: `userId → customer → property → policy → claim / payment`. An ADMIN bypasses every check. "List everything" endpoints became admin-only, so `GET /claims/mine` was added to give customers their own view.

§3 Blocker detail

1 A customer could approve their own claim

Cleared

WAS `PUT /claims/{id}/status` had no role guard. The gateway forwarded `X-User-Role`, but a repo-wide grep confirmed *nothing ever read it*. A plain CUSTOMER token returned `200` with `claimStatus: APPROVED`.

FIX `ClaimService.updateStatus` now calls `requireAdmin` before anything else. A claim already APPROVED or REJECTED can no longer be re-decided, which returns `409`.

EVIDENCE

```
CUSTOMER approves own claim   403 PASS
B approves A's claim          403 PASS
ADMIN sets IN_REVIEW          200 PASS
ADMIN approves                200 PASS
ADMIN re-decides settled      409 PASS
ADMIN, missing claim          404 PASS
```

2 Any user could read and overwrite any other user's data

Cleared

WAS Authentication happened, authorization did not. As user B, against user A's records: read A's PII, *overwrote A's profile* (name became "HACKED BY B"), read A's policy, downloaded A's policy PDF, listed every customer's PII and every user's notifications — all `200`.

FIX Ownership enforced in every service via the chain in §2. Single-record reads and writes require owner-or-admin; list-everything endpoints require ADMIN. `POST /customers` and `POST /properties` now take the owner from the token and ignore the body's `userId` / `customerId` for non-admins, so a profile cannot be created for someone else.

EVIDENCE

```
B reads A's customer          403 PASS
B overwrites A's customer     403 PASS
B reads A's property/policy   403 PASS
B downloads A's PDF           403 PASS
B claims on A's policy        403 PASS
B pays / lists A's payments   403 PASS
B reads A's claim / inbox     403 PASS
A lists all customers         403 PASS
ADMIN lists all customers     200 PASS
A reads own records           200 PASS
```

3 Business services were wide open on their own ports

Cleared

WAS Only the gateway checked JWTs, and nothing forced traffic through it. `GET :8082/customers`, `:8083/policies`, `:8084/claims` and `:8085/notifications` each returned `200` with no token whatsoever.

FIX Each service has a `GatewayAuthFilter` (a plain `OncePerRequestFilter` — no new dependency) requiring `X-Gateway-Auth` to match its configured secret, compared in constant time via `MessageDigest.isEqual`. Feign interceptors forward the header so inter-service calls still work.

EVIDENCE

```
:8082 / :8083 / :8084 / :8085 direct   401 PASS
direct + forged X-User-Role: ADMIN     401 PASS
direct + guessed gateway secret        401 PASS
direct + correct secret, CUSTOMER      403 PASS
gateway, no token / bad token          401 PASS
gateway, /auth/health (public)         200 PASS
```

4 Every auth-service exception came back as 401

Cleared

WAS auth-service was the only service with no `@RestControllerAdvice`. An exception in the controller escaped to the servlet container's `/error` forward, which `SecurityConfig` then blocked because it permitted only `/auth/**` with `anyRequest().authenticated()` — so the entry point converted it to a bare `401`. A duplicate email, an invalid role and a missing field were all reported as "Unauthorized".

FIX Added a handler with `DuplicateResourceException → 409`, `BadRequestException → 400`, validation and malformed-JSON mapping, and `.requestMatchers("/error").permitAll()` so the real response can actually reach the client. Added the missing `spring-boot-starter-validation` and constraints on both DTOs. Also removed a leftover `System.out.println("REGISTER API CALLED")` and a redundant second password check that ran after `AuthenticationManager` had already verified it.

EVIDENCE

```
duplicate email     409 PASS   role=SUPERUSER   400 PASS
missing fullName    400 PASS   empty body {}    400 PASS
malformed JSON      400 PASS   short password   400 PASS
wrong password      401 PASS   valid login      200 PASS
```

5 A duplicate profile broke customer lookup permanently

Cleared

WAS `customers.user_id` had no unique constraint, but `findByUserId` returned `Optional`. Creating two profiles for one user made `GET /customers/user/{id}` return `500` from then on, with no way to recover through the API.

FIX Two layers: `existsByUserId` in the service rejects a second profile with `409`, and a database unique constraint closes the race between two concurrent requests that both pass the check.

EVIDENCE

```
A creates 2nd profile          409 PASS
A spoofs userId in body        409 PASS  (body ignored)
SHOW INDEX ... user_id         Non_unique = 0 PASS
```

WORTH KNOWING — THE ANNOTATION ALONE WAS NOT ENOUGH

Adding `unique = true` to the entity did **not** create the constraint. `ddl-auto=update` will not retrofit a unique index onto a column that already exists, so it was applied directly:

```
ALTER TABLE customers ADD CONSTRAINT uk_customers_user_id UNIQUE (user_id);
```

A fresh database picks it up from the annotation. This is precisely the gap a migration tool exists to close — adding Flyway or Liquibase is the durable fix.

6 Bad enum and type values returned 500 instead of 400

Cleared

WAS `valueOf()` was called on unvalidated input, and the catch-all `@ExceptionHandler(Exception.class)` mapped the resulting `IllegalArgumentException` to `500`. Affected `riskFactor`, `propertyType`, `?status=` (including correct-but-lowercase values) and any mistyped path variable.

FIX Enums parsed through small helpers that throw `BadRequestException` naming the allowed values; `?status=` taken as a String and parsed in the service; `MethodArgumentTypeMismatchException` and `HttpMessageNotReadableException` mapped to `400` in all four services. Construction year is now range-checked too.

EVIDENCE

```
riskFactor=EXTREME    400 PASS   propertyType=CASTLE     400 PASS
GET /policies/abc     400 PASS   GET /customers/abc      400 PASS
constructionYear=2030 400 PASS   ?status=BOGUS           400 PASS
?status=submitted     200 PASS  (lowercase now accepted)
```

7 Claims had no business rules at all

Cleared

WAS `submitClaim` checked only that the policy *existed*. All of these returned `201`: an incident dated 2099, an incident 36 years before the policy started, a loss of ₹99,999,999 against ₹1,000,000 of cover, and the same claim submitted three times.

FIX The Feign DTO was widened to carry the policy's status, start date, end date and coverage amount, and `validateAgainstPolicy` now rejects: a non-ACTIVE policy, a future incident date, a date outside the policy period, a loss above cover, and — via a repository check — a duplicate of the same policy + incident type + incident date.

EVIDENCE

```
incidentDate in future     400 PASS
incidentDate pre-policy    400 PASS
loss above coverage        400 PASS
unknown policy number      400 PASS
duplicate same incident    409 PASS
valid claim                201 PASS
```

8 Payments had no business rules at all

Cleared

WAS Underpayment (₹1 against a ₹6,000 premium), overpayment (₹99,999,999) and unlimited duplicate payments all returned `201`. Nothing related the amount to what was owed.

FIX `recordPayment` computes outstanding premium as *annual premium − sum of payments to date*, rejects anything above it with `400`, and rejects any payment against a fully-settled policy with `409`. Reading a policy's payments now runs the same ownership check as reading the policy.

EVIDENCE

```
Premium on the test policy = 35,000.00
overpay 99,999,999         400 PASS
part-pay 15,000            201 PASS
then overpay 25,000        400 PASS
then exact 20,000          201 PASS
then pay again             409 PASS
```

9 Policy and claim numbers could collide — Cleared

WAS Both were a random 6-digit suffix written to a `unique` column with no collision handling, so a clash surfaced as a random `500`. Claim numbers also lacked the year prefix policies had, leaving the whole space at one million — roughly even odds of a collision by ~1,200 claims.

FIX Both generators check `existsBy…` and redraw, up to ten attempts. Claim numbers gained the year prefix, matching the policy format.

EVIDENCE

```
6 policies issued back to back   6/6 unique PASS
claim number format             CLM-2026-119346 PASS
```

§4 Behaviour changes — check the frontend

Closing these blockers necessarily changed some responses. Any client built against the old behaviour needs these six adjustments.

#	CHANGE	WHAT TO DO
1	**New required environment variable**	`GATEWAY_SHARED_SECRET`, identical across the gateway and all four business services. Nothing starts without it.
2	`userId` / `customerId` in request bodies are ignored	On `POST /customers` and `POST /properties` the owner comes from the token. Both fields are now optional and only meaningful for an ADMIN acting on someone's behalf.
3	List endpoints are ADMIN-only	`GET /customers`, `/properties`, `/policies`, `/claims`, `/notifications` now return `403` for customers. Use the new `GET /claims/mine` and `GET /notifications/recipient/{own-email}`.
4	`POST /auth/register` returns `201`	Was `200`. Update any strict status check.
5	Claim number format changed	Now `CLM-2026-119346`; was `CLM-594280`. Widen any fixed-length parsing.
6	A 30-day claim reporting window is enforced	Taken from the policy PDF's own terms. This is the one rule inferred rather than specified — easy to relax via `REPORTING_WINDOW_DAYS` in `ClaimService`.

UNCHANGED AND RE-VERIFIED

The premium formula still produces **risk factor 1.4 / ₹35,000.00** on the README's worked example (house, built 2000, ₹50,00,000), and the boundary cases still hold — 20 years old is not "old", 21 is. Policy PDF generation still works end to end through Feign, returning a valid `%PDF` document. Error responses were also checked to confirm they no longer echo internal exception messages:

```
{"error":"Conflict","message":"An account with email … already exists","status":409}
{"error":"Bad Request","message":"Invalid role 'NOPE'. Allowed: CUSTOMER, ADMIN","status":400}
{"error":"Bad Request","message":"Malformed or unreadable request","status":400}
```

§5 Running it

```
# Windows PowerShell — all four are required
$env:MYSQL_ROOT_PASSWORD   = "your-mysql-password"
$env:JWT_SECRET            = "a-long-random-secret-at-least-32-bytes-please"
```

```
$env:JWT_EXPIRATION        = "3600000"
$env:GATEWAY_SHARED_SECRET = "another-long-random-secret"
```

`JWT_SECRET` must be identical for auth-service (signs) and api-gateway (verifies). It is base64-decoded before use, so it must decode to at least 32 bytes or HS256 signing fails at startup. `GATEWAY_SHARED_SECRET` must be identical for the gateway and all four business services. Start order is unchanged: Eureka (8761), then the gateway (8080), then the services (8081–8085).

TEST DATA CLEANUP

The audit's own test rows were deleted from the four business databases — every row in them was created by the smoke tests (oldest row 19:27:43 on 8 Sep, exactly when they began), and the duplicate `user_id` rows were blocking the constraint in blocker 5. The four original users (`revanth@` , `vadivel@` , `dheeru@` , `test@`) were left untouched.

§6 Still open — none of these are blockers

- jjwt 0.11.5 deprecated API (`signWith(alg, String)` , `setSigningKey(String)`) — works, warns on build.
- `PasswordHash.java` still ships a `main()` with a hardcoded password in production source.
- 79 build artifacts remain committed, including `.class` files and a packaged jar; only auth-service has a `.gitignore` .
- Test coverage is still a single empty `contextLoads()` . `PremiumCalculator` and the new `validateAgainstPolicy` are pure functions and the natural place to start.
- `POST /notifications` accepts any authenticated caller, since claim-service invokes it with the customer's own identity. Documented in the README.
- No schema migration tool, which is what made blocker 5 need a manual `ALTER TABLE` .
- A stray `package-lock.json` sits in auth-service — a Node lockfile in a Java module.
- Policy status never changes when a premium is fully paid; there is no `PENDING_PAYMENT` state to move to.

§7 What changed, by module

MODULE	FILES	PRINCIPAL CHANGES
`api-gateway`	3 mod	Forwards `X-User-Id` and `X-Gateway-Auth` ; strips client-supplied identity headers; open-path matching changed from prefix to exact.
`auth-service`	7 mod · 3 new	`userId` claim in the JWT; exception handler; `/error` permitted; validation starter and DTO constraints; debug print and redundant password check removed.
`customer-service`	6 mod · 6 new	Gateway filter; caller context; ownership on all reads/writes; duplicate-profile guard; unique `user_id` ; safe enum parsing.
`policy-service`	8 mod · 7 new	Gateway filter; Feign identity propagation; `OwnershipService` resolving user → customer; payment rules; collision-safe policy numbers; construction-year validation.
`claim-service`	7 mod · 5 new	ADMIN guard on status changes; claim business rules; ownership by customer email; `/claims/mine` ; widened policy DTO; collision-safe claim numbers.
`notification-service`	2 mod · 4 new	Gateway filter; ADMIN-only list; recipient-scoped inbox; its first exception handler.
`eureka-server`	—	No changes required.